

EMP ID: 299164

Project 1: Contract Testing

Objective

To verify that the Storefront and Catalog Service can communicate correctly without breaking each other's API.

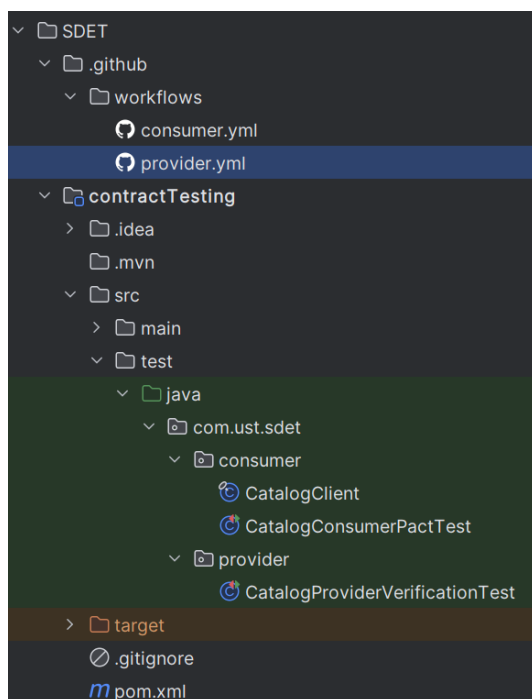
User Story

As a customer, I want the Storefront application to retrieve product information from the Catalog Service so that I can view product details correctly.

Overview

The consumer creates a contract based on its expectations. The contract is published to PactFlow. The provider downloads the contract, verifies it using WireMock, and publishes the verification results. Finally, Can-I-Deploy checks whether both applications are compatible before deployment.

Project Structure:



Test Scenarios

Scenario 1: Verify existing product (SKU-1)

- Request product details.
- Validate status code 200 and product information.

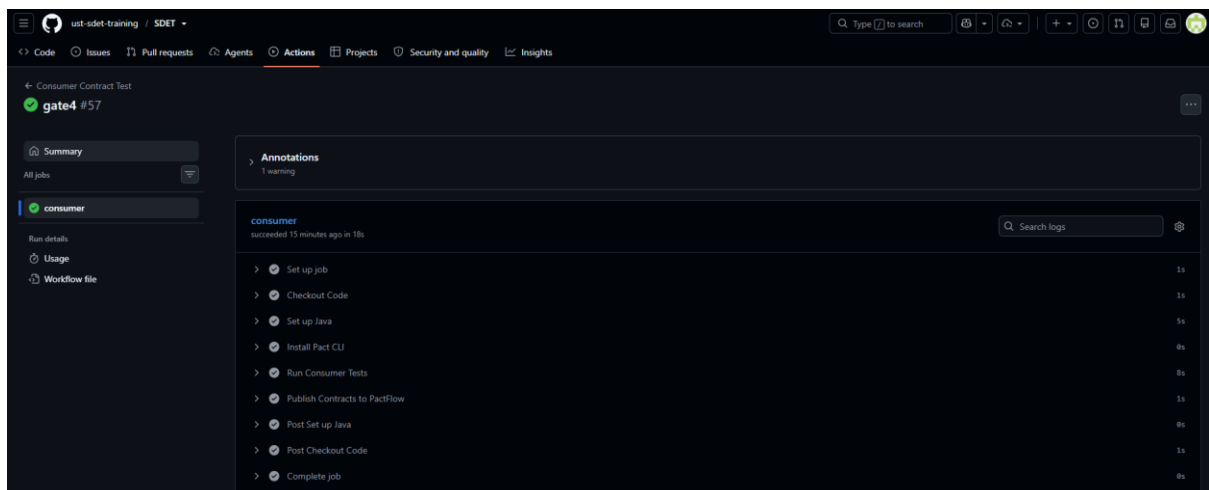
Scenario 2: Verify unavailable product (SKU-404)

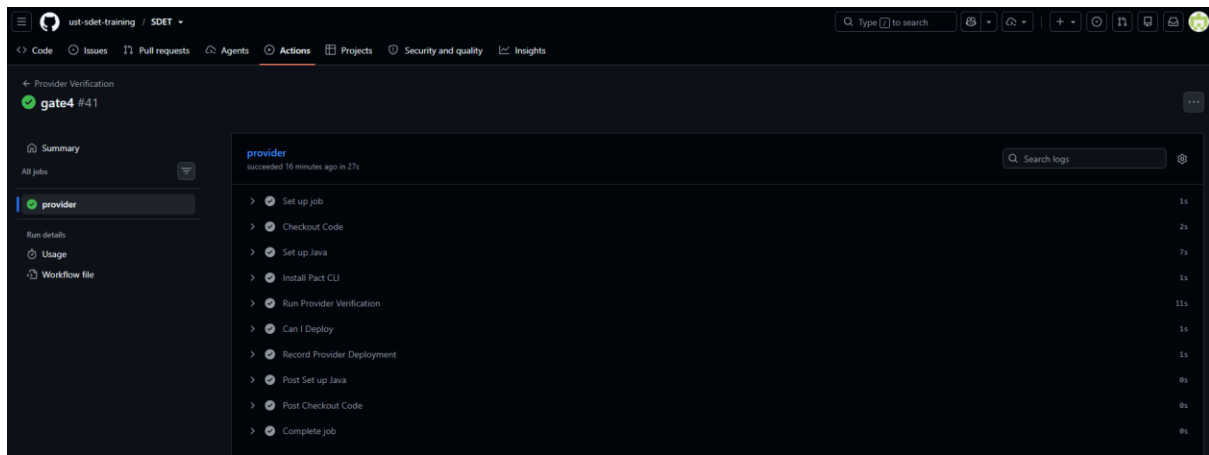
- Request a non-existing product.
- Validate status code 404 and error message.

Results

- Consumer contract generated successfully.
- Contract published to PactFlow.
- Provider verification completed.
- Deployment compatibility checked using Can-I-Deploy.

Evidence:





Conclusion:

This project successfully demonstrated Consumer-Driven Contract Testing using Pact and PactFlow. The consumer and provider were verified against a shared contract, ensuring API compatibility before deployment. Integrating the tests with GitHub Actions helped automate the contract testing process and improved confidence in service integration.

Project 2: Selenide UI Testing

Objective

To automate the product catalog workflow using Selenide and validate that users can search for products, navigate to the product details page, and verify product availability.

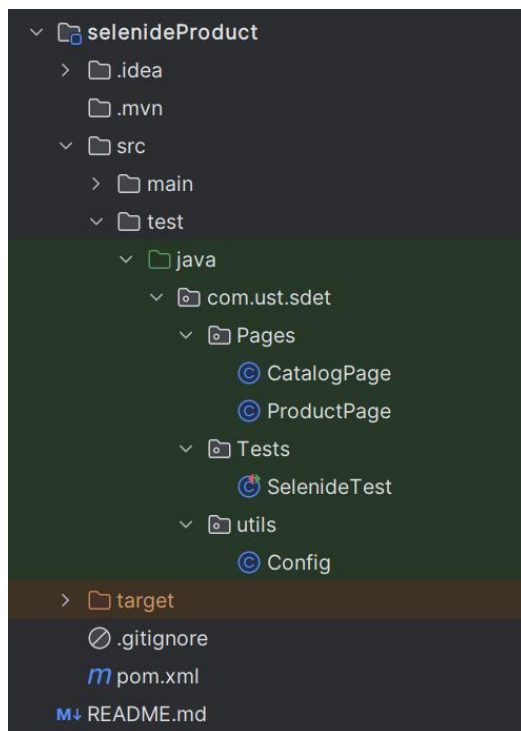
User Story

As a customer, I want to search for a product from the catalog and view its details so that I can verify its availability before purchasing.

Overview

This project implements UI automation using the Selenide framework and the Page Object Model (POM) design pattern. The test opens the product catalog, searches for a product, navigates to the product details page, and verifies that the product is available for purchase.

Project Structure:



Test Scenario

Scenario: Verify Product Availability

Steps:

1. Open the Product Catalog page.
2. Verify that the catalog page is displayed.
3. Search for a product using the keyword "trave".
4. Verify that matching products are displayed.
5. Open the first product from the search results.
6. Verify the Product Details page.
7. Verify that the Add to Cart button is enabled.
8. Verify that the availability status is "In stock".

Results

- Product catalog loaded successfully.
- Product search completed successfully.
- Product details page opened successfully.
- Product availability verified successfully.
- UI validations passed.

Conclusion:

This project successfully automated the product search and verification workflow using the Selenide framework. The test validated the catalog page, product search, navigation to the product details page, and product availability. Using the Page Object Model and Selenide's built-in waits resulted in a simple, maintainable, and reliable UI automation solution.